claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_4fd1a4ea-6c0\agent-aa847ab566fc0b78e.jsonl`
- SHA-256: `ac95f5909adffbd787d3f8ec1080dbf18f2657cbee52b1cae27b7ee505820199`
- Source modified: `2026-07-02T09:32:24+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_4fd1a4ea-6c0`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`

Transcript

1. user (2026-07-02T09:29:46.008Z)

ADVERSARIAL VERIFIER. A reviewer of the P23 main.py->orchestration.py extraction claims:

TITLE: Worktree mutated mid-review: three extra _main-seam deltas beyond the enumerated list (verification claims may predate final state)
SEVERITY: minor
DETAIL: backend/orchestration.py changed between my first and second read of it during this review. The first snapshot had module-level imports of VideoCompiler, emit_indexer_report, and compute_video_id and called them as bare locals inside the moved functions; the current state removes those three imports and routes all three through the call-time seam instead (_main.compute_video_id at orchestration.py:386, _main.emit_indexer_report at :571, _main.VideoCompiler at :702, with the second lazy 'import backend.main as _main' at :679 in _execute_compile). These are AST-visible deltas NOT in the review brief's enumerated list of (1)-(3), so the 'AST-identical, machine-verified' claim refers to a different snapshot than what I reviewed. The current form is import-mechanics-correct (the call-time _main import at :355/:679 executes before every _main.<name> use, and main.py:61/64/74 still binds VideoCompiler/emit_indexer_report/compute_video_id so the attribute lookups resolve), and I re-verified the final state directly: ruff clean, re-export identity holds for all 10 names, both import orders work. Action: re-run the AST-identity/ratchet verification against the final tree before merging, and update the PR's enumerated-delta list to include these three.
EVIDENCE: backend/orchestration.py:386, backend/orchestration.py:571, backend/orchestration.py:679, backend/orchestration.py:702; retained bindings at backend/main.py:61,64,74

REFACTOR UNDER REVIEW (uncommitted, in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs, branch refactor/p23-orchestration-split off origin/master):
- NEW backend/orchestration.py (739 lines): 10 members moved VERBATIM from backend/main.py:
  _finalize_reingest, _report_config_for_input, _ingest_remote_segments, _cloud_available,
  _maybe_dispatch_remote, _execute_processing, _remote_input_size_bytes, _CompileError,
  _resolve_compile, _execute_compile.
- backend/main.py 2360 -> 1703 lines; re-exports all 10 names via "from backend.orchestration import (...)" (noqa F401);
  5 now-unused imports removed (csv, math, Tuple, StitchingConfig, localize_watermark).
- Enumerated intentional deltas (everything else is AST-identical, machine-verified):
  (1) _maybe_dispatch_remote: "req: ProcessRequest" -> string forward-ref "ProcessRequest" (TYPE_CHECKING import).
  (2) _execute_processing: inserted "import backend.main as _main" after docstring; global_config/db_instance/
      ProcessRequest resolved via _main.<name> at call time (the job_worker seam convention).
  (3) two "# type: ignore[...]" comments (AST-invisible).
- NEW tests/test_main_module_budget.py: line budget 1750, route ceiling 18, re-export identity check.

```
- Verified already: AST-identity (10 moved + 51 retained members), ruff+format clean, mypy green
(142 files),
  ratchet test passes. Full suite running separately.
Inspect with: git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" diff  (and
git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" status); the new files are
untracked/modified in the worktree.
READ-ONLY: do not modify the worktree, do not checkout/commit.

Independently verify in the worktree (READ-ONLY). Default skeptical: real=true ONLY if you can
reproduce the evidence AND the failure scenario is concrete (a test that would fail, a runtime
path that breaks, an operational regression). A theoretical concern with no concrete consumer =
real=false.
```

2. user (2026-07-02T09:29:59.775Z)

```
 M backend/main.py
?? backend/orchestration.py
?? tests/test_main_module_budget.py
refactor/p23-orchestration-split
30736ab docs: make AUDIT_REPORT the umbrella code-audit doc; fold + archive the rest (#430)
```

3. user (2026-07-02T09:30:08.300Z)

```
    # consumers (hash / validators / segmenter) use the resolved local path.
    local_video = ""
    try:
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
        video_id = _main.compute_video_id(local_video)
    except Exception:
        storage.cleanup_acquired_local_input(
            req.video_path, local_video, getattr(config, "storage", None)
        )
            (response or {}).get("status") if isinstance(response, dict) else None,
        )
        # Always emit the per-video observability report (next to the video, or to
        # report.output_dir). Never raises. Surface the paths in the response. (The compute-path
        # proof lives on the job result + the [COMPUTE] log, not the report ? obsreport's
record_run
        # doesn't surface arbitrary run signals.)
        paths = _main.emit_indexer_report(
            db=db,
            video_id=video_id,
            # Resolved LOCAL path, not the raw URI: obsreport writes the report next to the
            # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://, local://)
    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel,
    and return the {status, filepath, download_url} result the SPA polls for. Returns a
    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a stitch
error)
    so the SPA always gets a clean verdict instead of a worker-crash error."""
    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

    notify = progress or (lambda stage: None)
    params = json.loads(job.get("params") or "{}")
    video_id = job.get("video_id") or ""
    segment_ids = params.get("segment_ids") or []
    locale = params.get("locale")

    notify("resolving")
    try:
```

```
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, params.get("output_filename", "")
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    output_dir = (
        getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
    )
    # Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
job);
    # unknown/absent locale keeps the configured default unchanged.
    localized_watermark = localize_watermark(stitching.watermark, locale)
    compiler = _main.VideoCompiler(
        output_dir,
        begin_padding=stitching.begin_padding,
        end_padding=stitching.end_padding,
```

4. user (2026-07-02T09:30:10.071Z)

```
11:the ``backend.main`` module object at call time (``import backend.main as _main``), never as
a
13:...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working unchanged.
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
356:    import backend.main as _main  # call-time seam: main's globals/model stay patchable
362:        if req is None and db is not None and isinstance(db, _main.ProcessRequest):
365:        elif req is None and isinstance(config, _main.ProcessRequest):
370:        config = _main.global_config
372:        db = _main.db_instance
386:        video_id = _main.compute_video_id(local_video)
571:        paths = _main.emit_indexer_report(
679:    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
702:    compiler = _main.VideoCompiler(
```

5. user (2026-07-02T09:30:30.940Z)

```
343: def _execute_processing(
594: def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
615: class _CompileError(Exception):
625: def _resolve_compile(
674: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
)
from backend.config import (  # new typed config
    load_config as load_app_config,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.validators.base import registry
from backend.providers import provider_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure  # standardized error/result contract
from backend.storage.capabilities import (
    storage_capabilities,  # P3: secret-free "which media can this box use" report
)
from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
    ffmpeg_bin,
    ffprobe_bin,
)
from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import (
    output_base,
```

```
    resolve_under_root,
    safe_output_filename,
    validate_input_path,
)
```

6. user (2026-07-02T09:30:32.762Z)

```
1271:        video_id = compute_video_id(local_video)
1456:        paths = emit_indexer_report(
1816:    compiler = VideoCompiler(
2094:        compiler = VideoCompiler(
2216:            video_id = compute_video_id(local_video)
2278:                paths = emit_indexer_report(
```

7. user (2026-07-02T09:31:12.555Z)

```
Exit code 1
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
FileNotFoundError: [Errno 2] No such file or directory: '/tmp/orig_main.py'
```

8. user (2026-07-02T09:31:26.818Z)

```
_finalize_reingest AST-identical
_report_config_for_input AST-identical
_ingest_remote_segments AST-identical
_cloud_available AST-identical
_maybe_dispatch_remote DIFFERS
_execute_processing DIFFERS
_remote_input_size_bytes AST-identical
_CompileError AST-identical
_resolve_compile AST-identical
_execute_compile DIFFERS
_execute_processing -> _main attrs: ['ProcessRequest', 'compute_video_id', 'db_instance',
'emit_indexer_report', 'global_config'] | lazy imports at: ['line 356']
_execute_compile -> _main attrs: ['VideoCompiler'] | lazy imports at: ['line 679']
_maybe_dispatch_remote -> _main attrs: [] | lazy imports at: []
module-level imports of the 3 names: []
```

9. user (2026-07-02T09:31:45.422Z)

```
maybe_dispatch delta is ONLY the forward-ref: True
```

10. user (2026-07-02T09:31:56.780Z)

```
re-export identity: True
main binds seam names: True
main-first import OK
orchestration-first import OK
```

11. user (2026-07-02T09:32:24.807Z)

```
Structured output provided successfully
```